

方法1：Trace下板调试

在前面的章节中，我们在虚拟机上使用Trace测试框架对CPU进行了功能上的仿真验证。在下板时，我们也可以使用Trace测试程序来调试，即将Trace比对测试的汇编程序导入IROM存储器并生成比特流下板运行。

本课程提供可在FPGA上运行Trace比对的汇编测试程序，该程序的源码见测试包的 `cdp-tests / asm / start.dump`。

关于下板运行Trace的说明

本节所介绍的下板运行Trace并非必做内容，而是一种在FPGA板上运行的调试手段。

miniLA无start.dump测试程序。如果需要下板测试Trace框架中的测试用例，则只能单个进行下板测试。

1. 导入测试程序

按照[流水线CPU实验步骤](#)，使用bin2coe.py脚本把 `cdp-tests / bin` 目录下的start.bin转换成.coe文件并导入 `bram_axi` 的IP核。

接着，运行综合、实现、生成比特流，最终下板运行Trace测试。

2. 测试结果说明

cdp-tests测试包提供了37条指令（不含乘除法指令）的测试程序。每通过一个测试点，数码管显示的数值会加1，直至测试完毕。测试完毕时，数码管将以16进制形式显示测试总数和通过了的测试数量。

举例说明

以miniRV为例，如果实现了21条必做指令并通过了测试，数码管将显示 `0x25000015`。数码管高2位显示 `0x25`，表示共有 37 个测试点，数码管低2位显示 `0x15`，表示通过了 21 个测试点。

需要注意的是，对于测试不通过的情形，如果数码管显示的值停在n，则表示第n+1条指令的测试失败。测试失败时，可查看start.dump的测试代码以定位出错指令，并进行相应的调试。

在下板测试中,如果数码管显示卡在某一个功能点,没有继续计数,一种方法是采用虚拟机中的Trace测试框架来进一步定位错误点。

- **Step1**: 进入 `cdp-test` 目录,输入 `make` 命令以重新编译;
- **Step2**: 输入 `make run TEST=start` 以运行start测试。

例如,某次测试时报错, `debug_wb_pc` 显示了 `0x000018f8`, 如下图所示。

```
===== Diffrence =====
SIGNAL NAME      REFERENCE      MYCPU
debug_wb_pc      0x000018f8      0x000018f8
debug_wb_ena      1              1
debug_wb_reg      10             10
debug_wb_value    0x000038f8      0x00002000
[diffptest] Test Failed!
```

打开start.dump文件,找到PC值为 `0x000018f8` 的指令,即可定位到具体出错的指令,如下图所示。

```
000018f8 <n25_auihc_test>:
18f8: 00002517      auipc  x10,0x2
18fc: 71c50513      addi   x10,x10,1820 # 4014 <tdat_sw2>
1900: 004005ef      jal   x11,1904 <n25_auihc_test+0xc>
```

显然,在上述例子中,mycpu执行到auipc指令时出错。

如果在虚拟机的Trace测试框架中通过了测试,但下板运行Trace测试程序时出错,即出现仿真和下板不一致的情况,请参考[常见问题汇总之3.6](#)。

方法2: FPGA在线调试

1. 下板前的调试

在下板验证之前, 务必对自己设计的CPU进行充分的仿真调试。此外, 还需认真排查综合和实现时的warning。一般地, Critical Warning是强烈建议要修正的, 普通的Warning则是尽量修正。

排查了Warning之后, 建议人工检查RTL代码, 避免以下问题:

- 多驱动
- 模块的输入/输出端口接入的信号方向有误
- 时钟复位信号接错
- 代码不规范, 乱用阻塞赋值和非阻塞赋值, 随意使用always语句
- 仿真时控制信号有不定态"X"或高阻"Z" (特别注意顶层接口上不要出现"X"和"Z")
- 时序违约
- 模块里的控制路径上的信号未进行复位

上述问题通常会导致仿真通过而下板出错的情况。如果出现仿真对, 下板不对, 请首先检查RTL代码中是否存在以上问题。

2. FPGA在线调试

在FPGA开发过程中, 经常碰到仿真上板行为不一致, 更多时候是仿真正常, 上板异常。由于上板调试手段薄弱, 导致很难定位错误。这时候可以借助Xilinx的下载线进行在线调试, 在线调试是在FPGA上运行过程中探测预定好的信号, 然后通过USB编程线缆显示到上位机上。

下面简单介绍使用在线调试的方法: RTL里设定需探测的信号, 综合并建立Debug core, 实现并生产bit流文件, 下载bit流和debug文件, 上板观察。

2.1 抓取需探测的信号

在RTL源码中, 给想要抓取的探测信号声明前增加 (`*mark_debug = "true"`)。

比如, 我们想要在FPGA板上观察debug信号、PC寄存器和数码管寄存器, 需要在代码里这样设置。

```
//debug interface
(*mark_debug = "true"*)output [31:0] debug_wb_pc;
(*mark_debug = "true"*)output [31:0] debug_wb_rf_wen;
(*mark_debug = "true"*)output [4:0] debug_wb_rf_wnum;
(*mark_debug = "true"*)output [31:0] debug_wb_rf_wdata;
```

抓取写回信息

```
wire inst_ex_dib;
(*mark_debug = "true"*)reg [31:0] inst_pc_r;
reg [31:0] inst_code_r;
```

抓取PC寄存器

```
4 reg [31:0] led_r0_data;
5 reg [31:0] led_r1_data;
6 (*mark_debug = "true"*) reg [31:0] num_data;
7 wire [31:0] switch_data;
8 wire [31:0] btn_key_data;
```

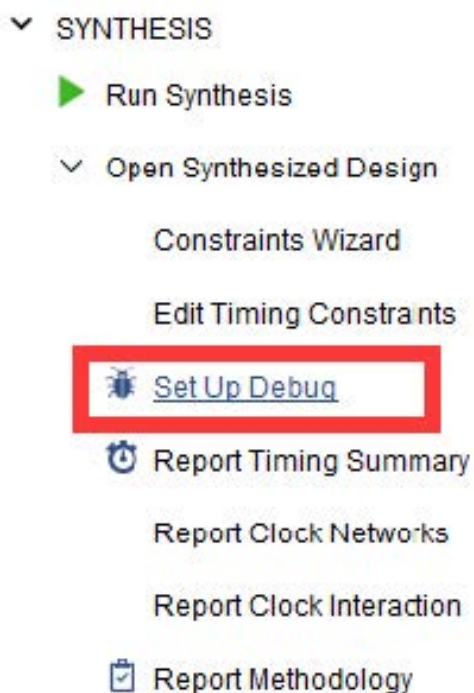
抓取数码管寄存器

设定完成后, 就可以运行综合了。

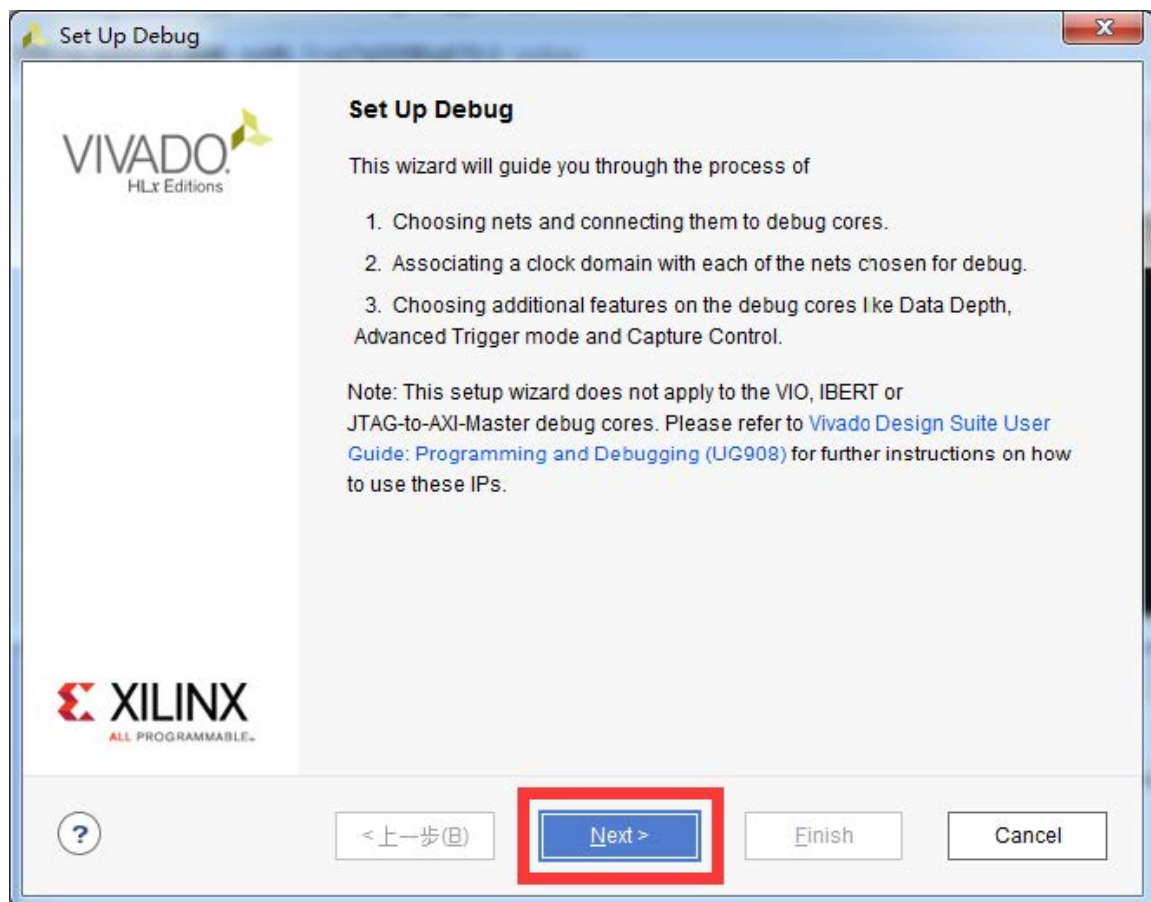
2.2 综合并建立debug

在综合完成后, 需建立debug。

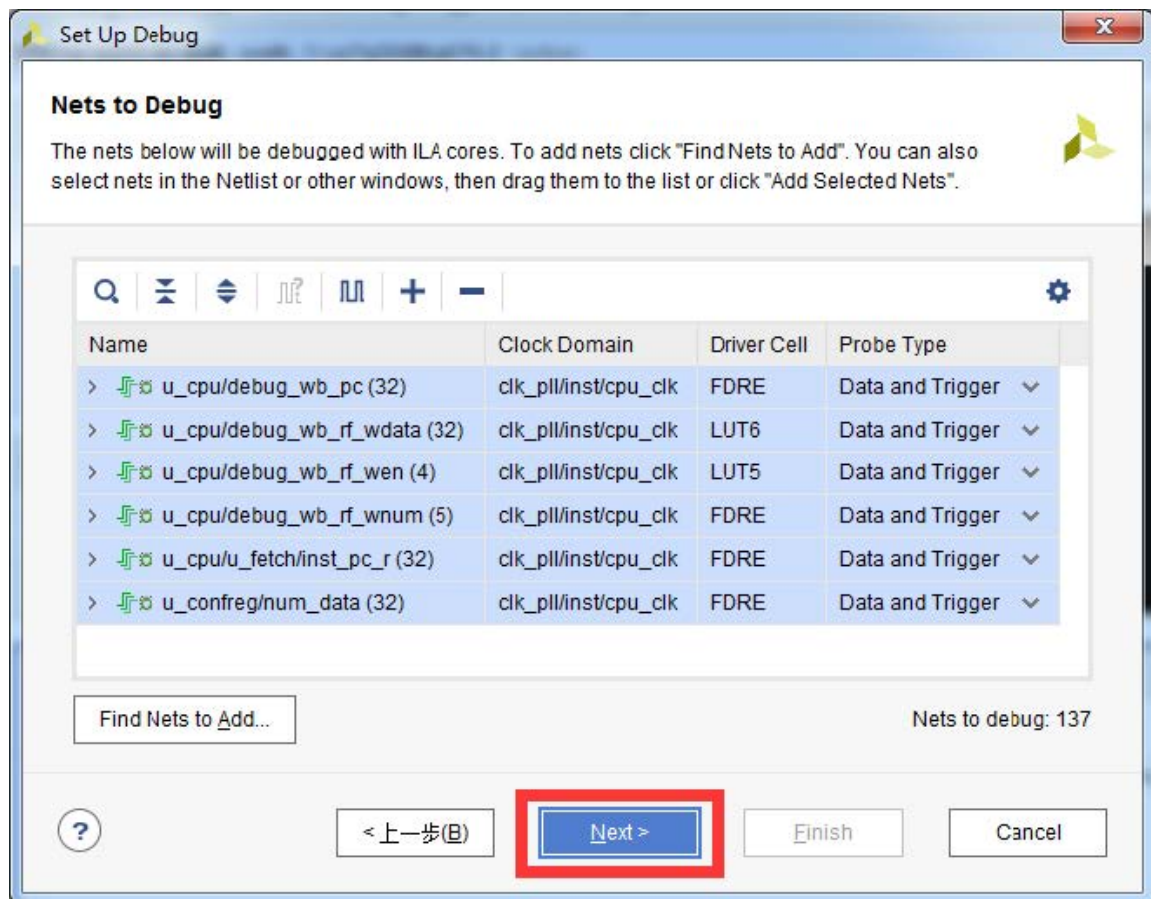
点击工程左侧synthesis->Open Synthesized Design->Set Up Debug。



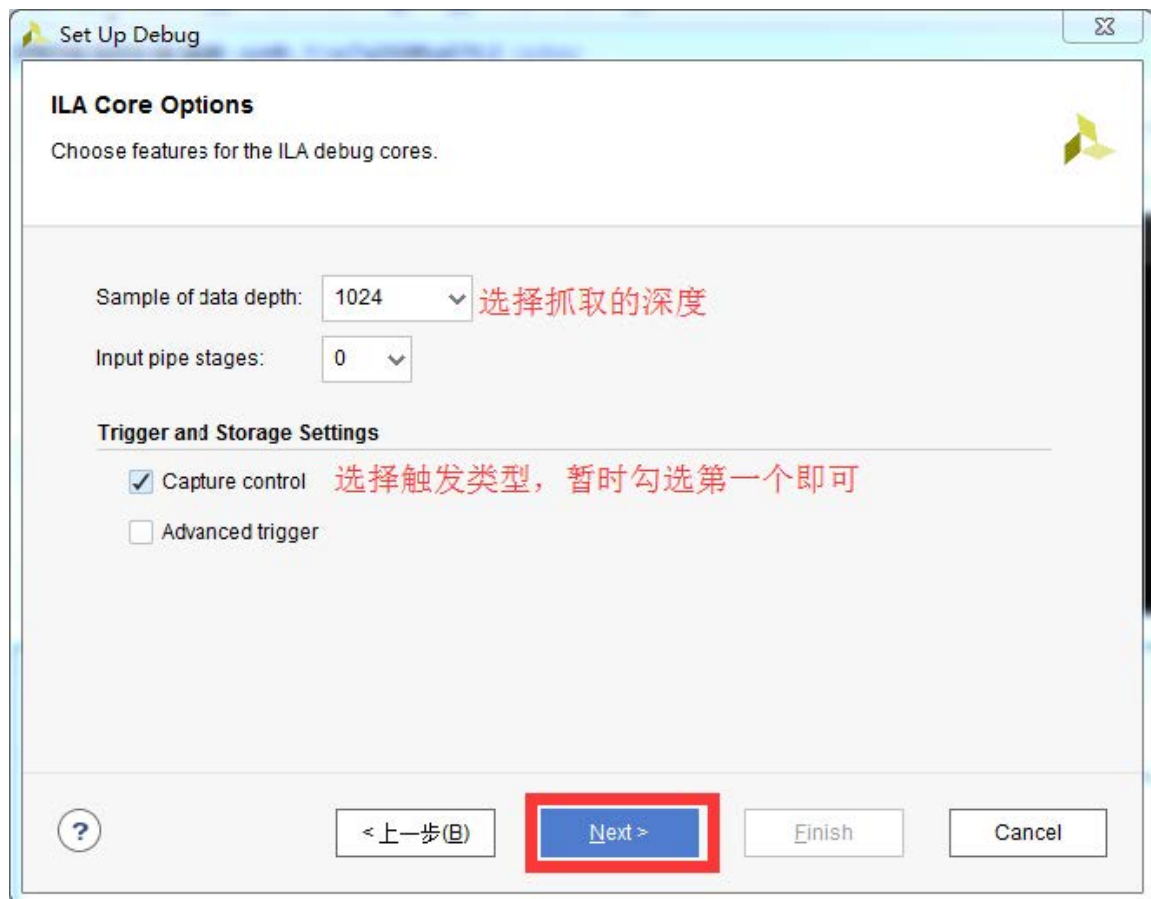
随后会出现如下界面, 点击Next:



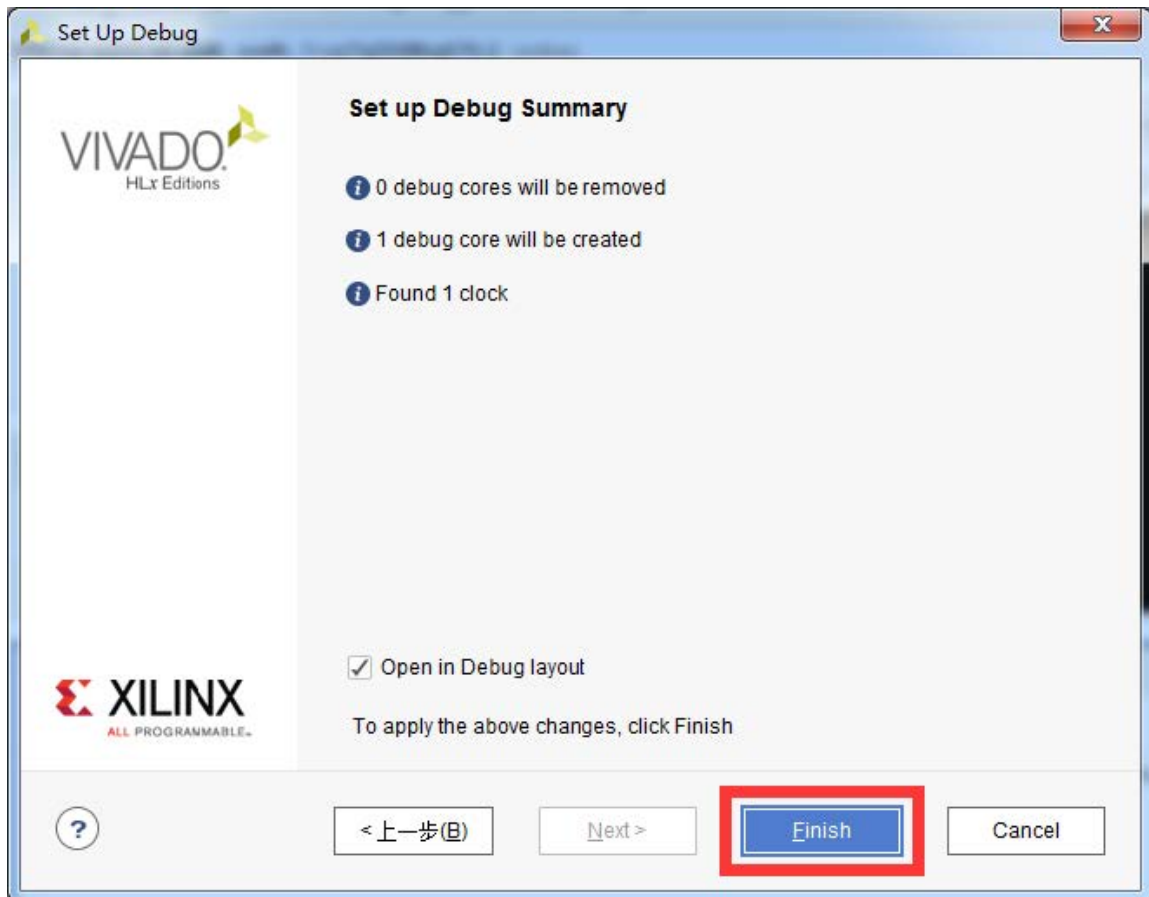
随后会列出抓取的Debug信息, 点击Next:



选择抓取的深度和触发控制类型，点击Next（更高级的调试可以勾选“Advanced trigger”）：

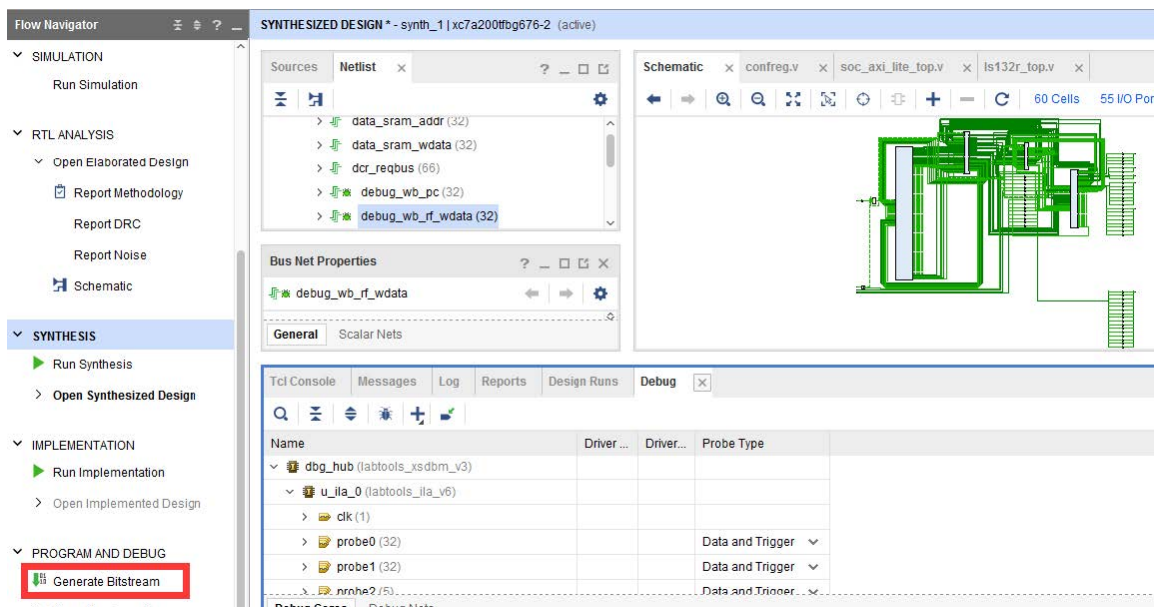


最后，点击Finish：

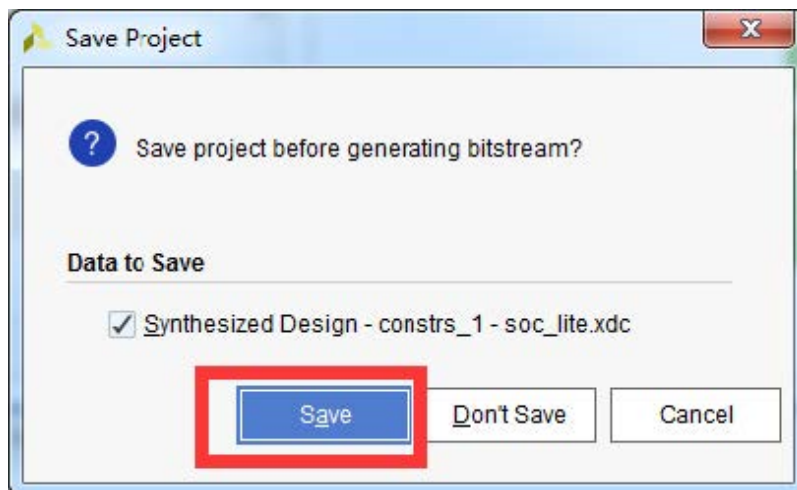


2.3 实现并生产bit流文件

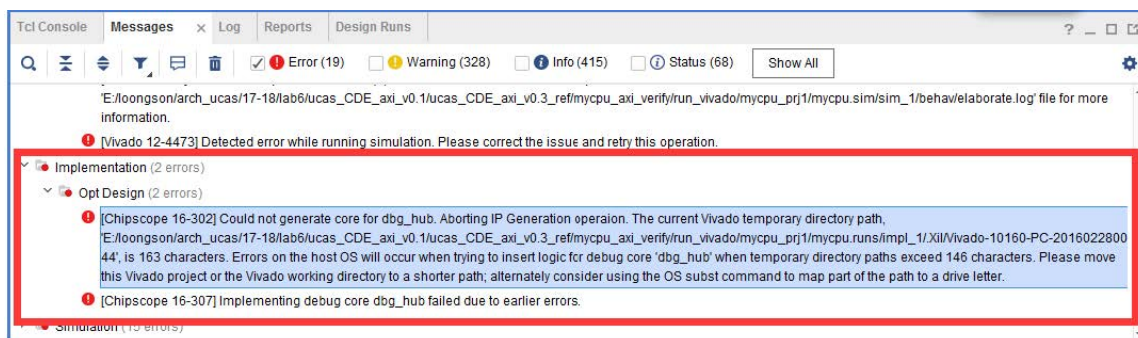
在2.2节完成后会出现类似下图界面，直接点击Generate Bitstream：



弹出如下界面，点击Save：

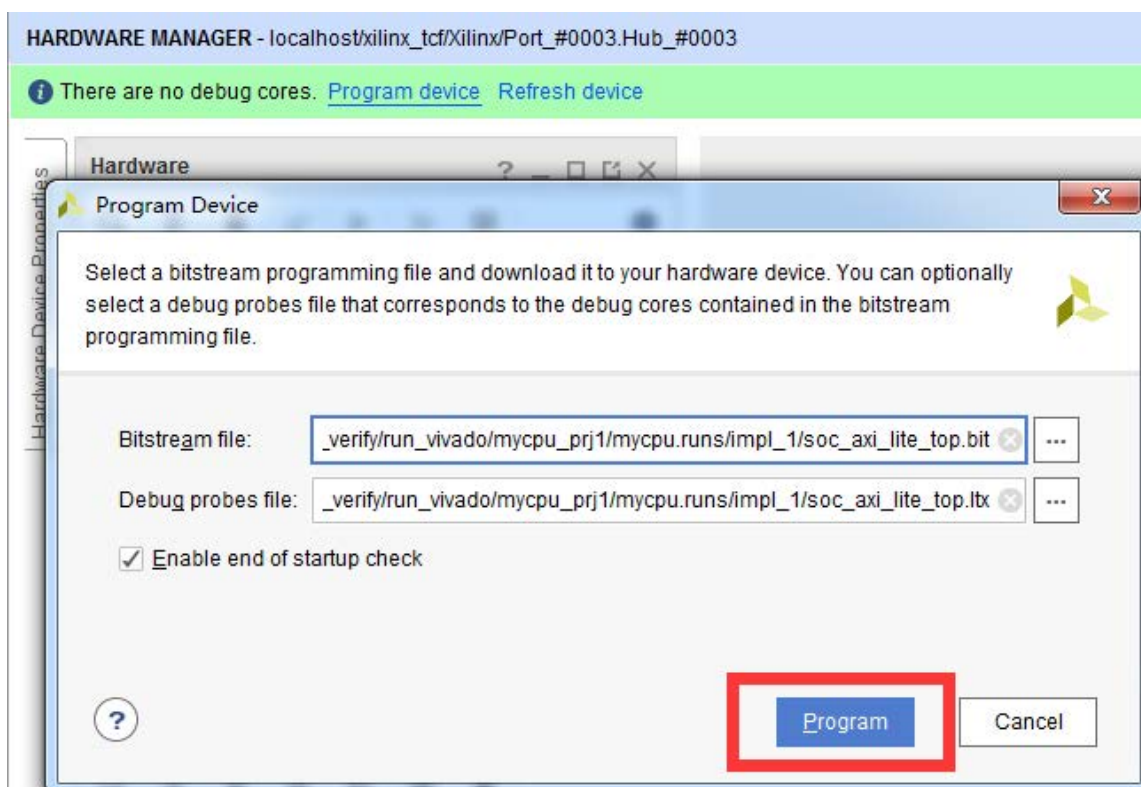


如果有后续弹出界面, 继续点击OK或Yes即可。这时就进入后续生产bit文件的流程了, 此时Vivado界面里的synthesis design界面就可以关闭了。如果发现以下错误, 则是因为路径太深, 引用起来名字太长, 降低工程目录深度即可:



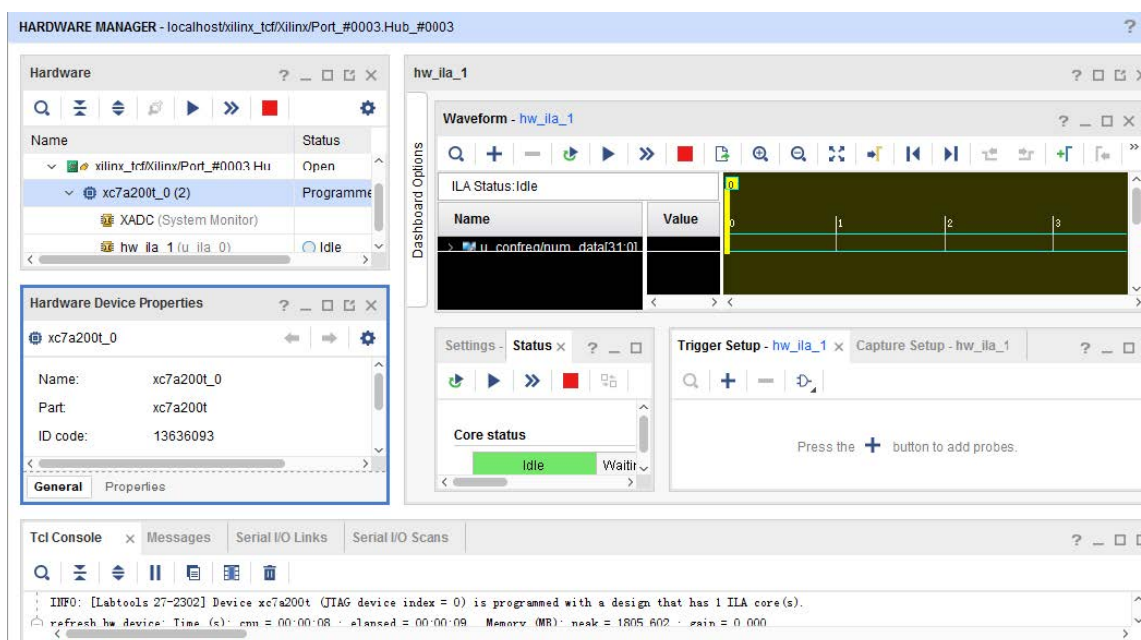
2.4 下载bit流和debug文件

在完成2.3节后, 会生成bit流文件和调试使用Itx文件。这里, 打开Open Hardware Manager, 连接好FPGA开发板后, 选择Program device, 如下图。自动加载了bit流文件和调试的Itx文件。选择Program, 等待下载完成。

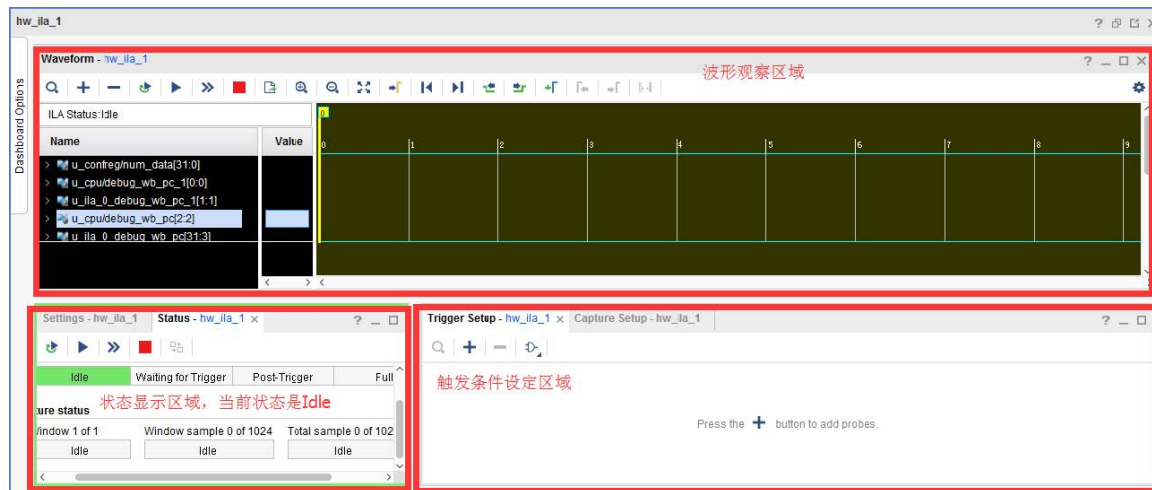


2.5 下板观察

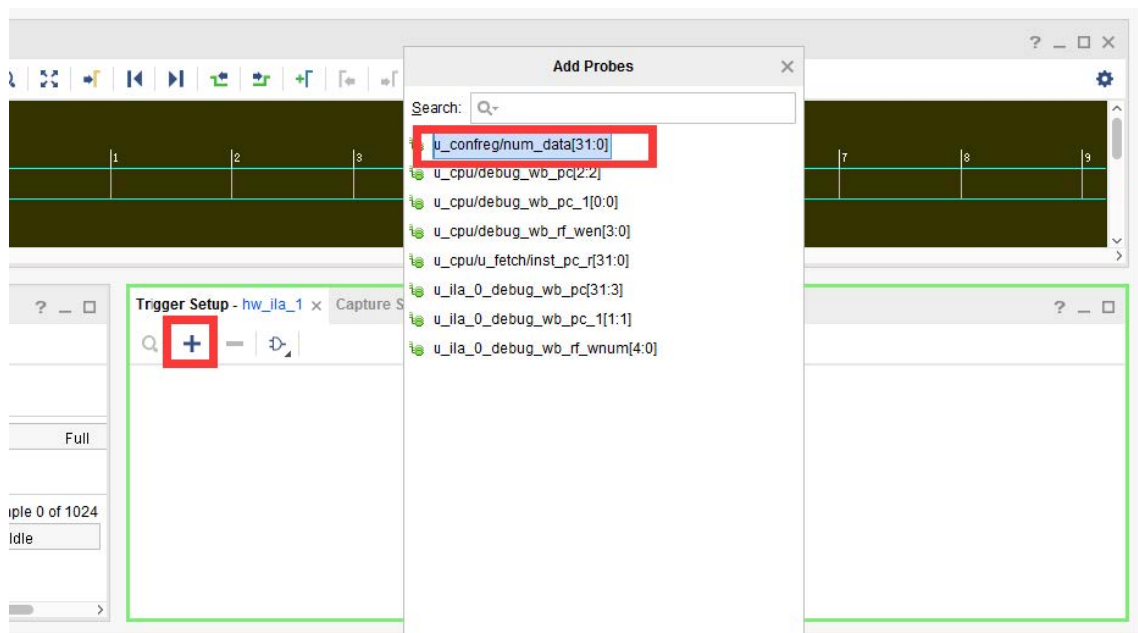
在下载完成后, vivado界面如下, 在线调试就是在hw_ila_1界面里进行的。



hw_ila_1界面主要分为3个界面, 分别如下:



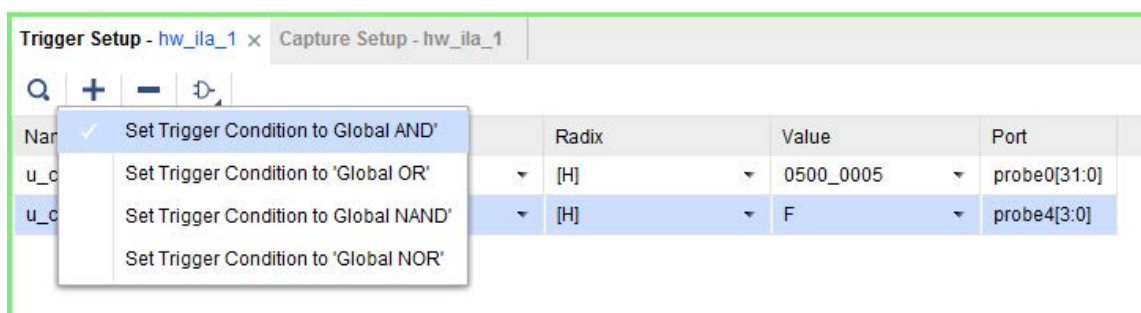
首先，我们需要在右下角区域设定触发条件。所谓触发条件，就是设定该条件满足时获取波形，比如我先设定触发条件是数码管寄存器到达0x0500_0005。在下图中，先点击“+”，在双击num_data。



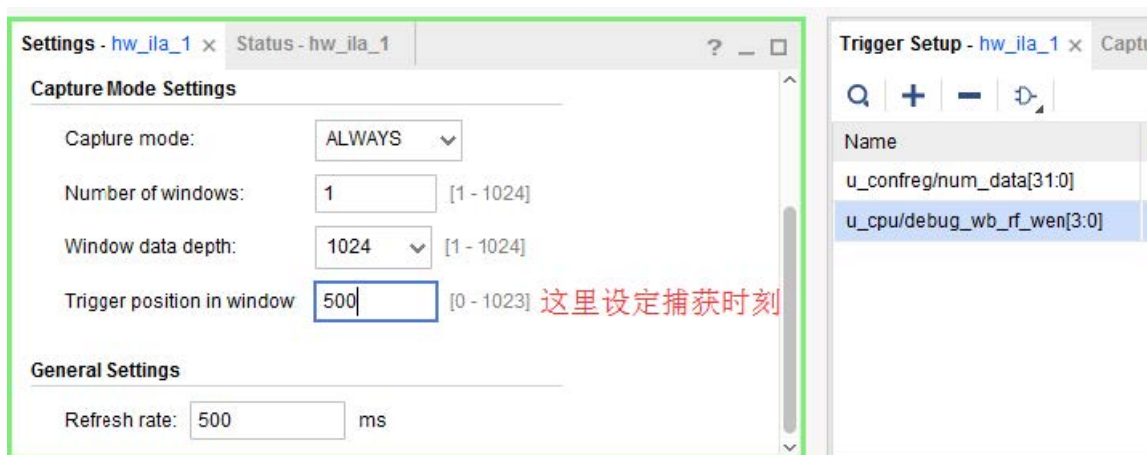
之后会出现如下界面，设定好触发条件。



可以设定多个触发条件, 比如, 再加一个除法条件是写回使能是0xf, 可以设定多个触发条件直接的关系, 比如是任意一个满足、两个都是满足等等, 如下图:



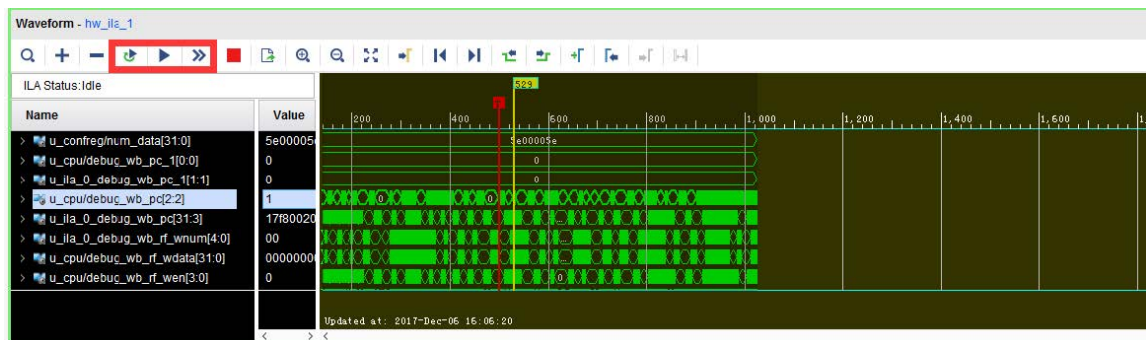
在左下角窗口, 选择settings, 可以设定Capture选项, 可能经常用到的是Trigger position in window, 用来设定触发条件满足的时刻在波形窗口的位置。比如, 下图设定为500, 当触发条件满足时, 波形窗口的第500个clk的位置是该条件, 言下之意, 将触发条件满足前的500个clk的信号值也抓出来了, 这样可以看到触发条件之前的电路行为。Refresh rate设定了波形窗口的刷新频率。



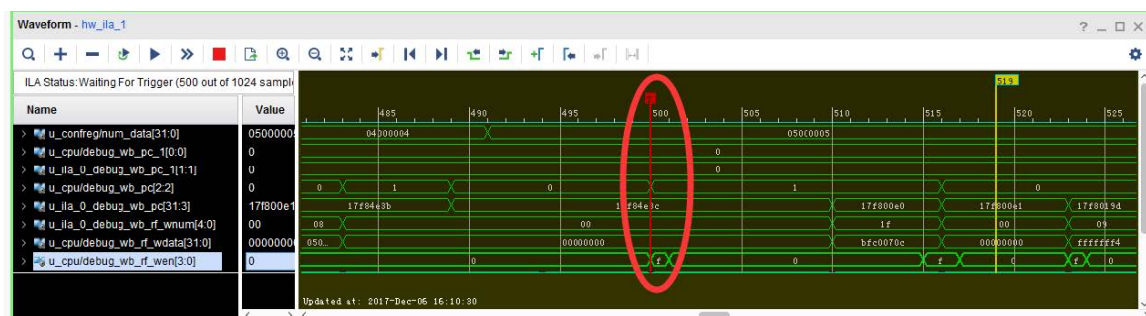
触发条件建立后, 就可以启动波形抓取了, 最关键的有三个触发按键, 即下图圈出的3个按键:

- 左起第一个, 设定触发模式, 有两个选项: 单触发; 循环触发。当该按键按下时, 表示循环检测触发, 那么只要触发条件满足, 波形窗口就会更新。当设置为单触发时, 就是触发一次完成后, 就不会再检测触发条件了。比如, 如果我们设定触发条件是PC=0xbfc00690, 那么如果该PC被多次执行到。如果设定为单触发, 那按下FPGA板上的复位键, 波形窗口只会展示第一次触发时的情况。如果设定循环触发, 那么波形窗口会以Refresh rate不停刷新新捕获的触发条件。
- 左起第二个, 等待触发条件被满足。点击该按键, 就是等待除法条件被满足, 展示出波形。
- 左起第三个, 立即触发。点击该按键, 表示不管触发条件, 立即抓取一段波形展示到窗口中。

下图就是点击第三个按键得到的波形，因为是立即触发，所以num_data不是0x0500_0005，且有一条标注为“T”的红色，就是触发的时刻。由于触发时刻位于波形窗口的500 clk位置，所以红色的位置正好是500 clk处。



从上图也能看到，num_data是0x5c00_005c，表示一次测试已经完成了。所以这时候，点击第二个触发按键等待触发，会发现波形窗口没有反应。这是因为触发条件没有被满足，这时按下FPGA板上的复位键即可。结果如下图。红色圈出的就是触发条件：
num_data==32'h5c00_005c && rf_wen==4'hf。



剩下的debug过程，就和仿真debug类似了，去观察波形。但在线调试时，你无法添加在2.1节未被添加debug mark的信号。在线调试过程中，可能需要不停的更换触发条件，不停的按复位键。

2.6 注意事项

在2.1节中添加要抓取的信号时，不要给太多信号标注debug mark了。在线调试时抓取波形是需要消耗电路资源和存储单元的，因而能抓取的波形大小是受限的。应当只给必要的debug信号添加debug mark。

在2.2节中抓取波形的深度，不宜太深。如果设定得太深，那么会存在存储资源不够，导致最后生成bit流和debug文件失败。

抓取的信号数量和抓取的深度是一对矛盾的变量。如果抓取深度相对较低，抓取的信号数量就可以相对多些。

在2.5节中，相对仿真调试，在线调试对调试思想和技巧有更高的要求，请好好整理思路，多多总结技巧。特别强调以下几点：

- 触发条件的设定有很多组合, 请根据需求认真考虑, 好好设计
- 通常只需要使用单触发模式, 但循环触发有时候也很有用, 必要时好好利用
- 在线调试界面里很多按键, 请自行学习, 可以网上搜索资料, 也可以在Xilinx官网上搜索, 查找官方文档等

最后再提醒一点, 仿真通过但上板失败时, 请先重点排查其他问题, 最后再使用在线调试的方法。也就是仿真通过, 上板异常时, 应按以下流程排查:

- 1) 复核生成、下载的bit文件是否正确;
- 2) 复核仿真结果是否正确;
- 3) 检查实现时的时序报告 (Vivado界面左侧“Timing Summary”);
- 4) 认真排查综合和实现时的Warning;
- 5) 排查RTL代码规范, 避免多驱动、阻塞赋值乱用;
- 6) 使用Vivado的逻辑分析仪进行板上在线调试。